🧭 Navigation2 Multi-Point Navigation (Waypoint Following)

1. Course Content

- Master using Navigation2 waypoint following (Waypoint Following / Navigate Through Poses) function to achieve multi-target point continuous autonomous cruising.
- Master the complete process of batch calibrating target waypoints in RViz2, selecting cruising mode, starting cruising task, and monitoring the robot passing through each waypoint in sequence.

---

2. Multi-Point Waypoint Navigation Principle Analysis

In actual robot application scenarios (such as warehouse inspection, fixed-route patrol, multi-station delivery), robots often need to continuously pass through multiple spatial stations. Navigation2 provides two mainstream multi-point cruising modes:

1. **Waypoint Following (Waypoint Following)**:
    - The robot treats the waypoint list as a series of independent stations.
    - The robot sequentially plans to travel to Point 1, accurately stops, and can trigger plugin tasks (such as waiting in place, grabbing, or taking photos), and after completion, automatically plans to Point 2, until all points are executed.
2. **Navigate Through Poses (Continuous Traversal)**:
    - The global planner concatenates all waypoints as one continuous compound global path.
    - The robot smoothly passes through intermediate waypoints at the set cruising speed without decelerating or stopping, and finally smoothly brakes at the end point.

---

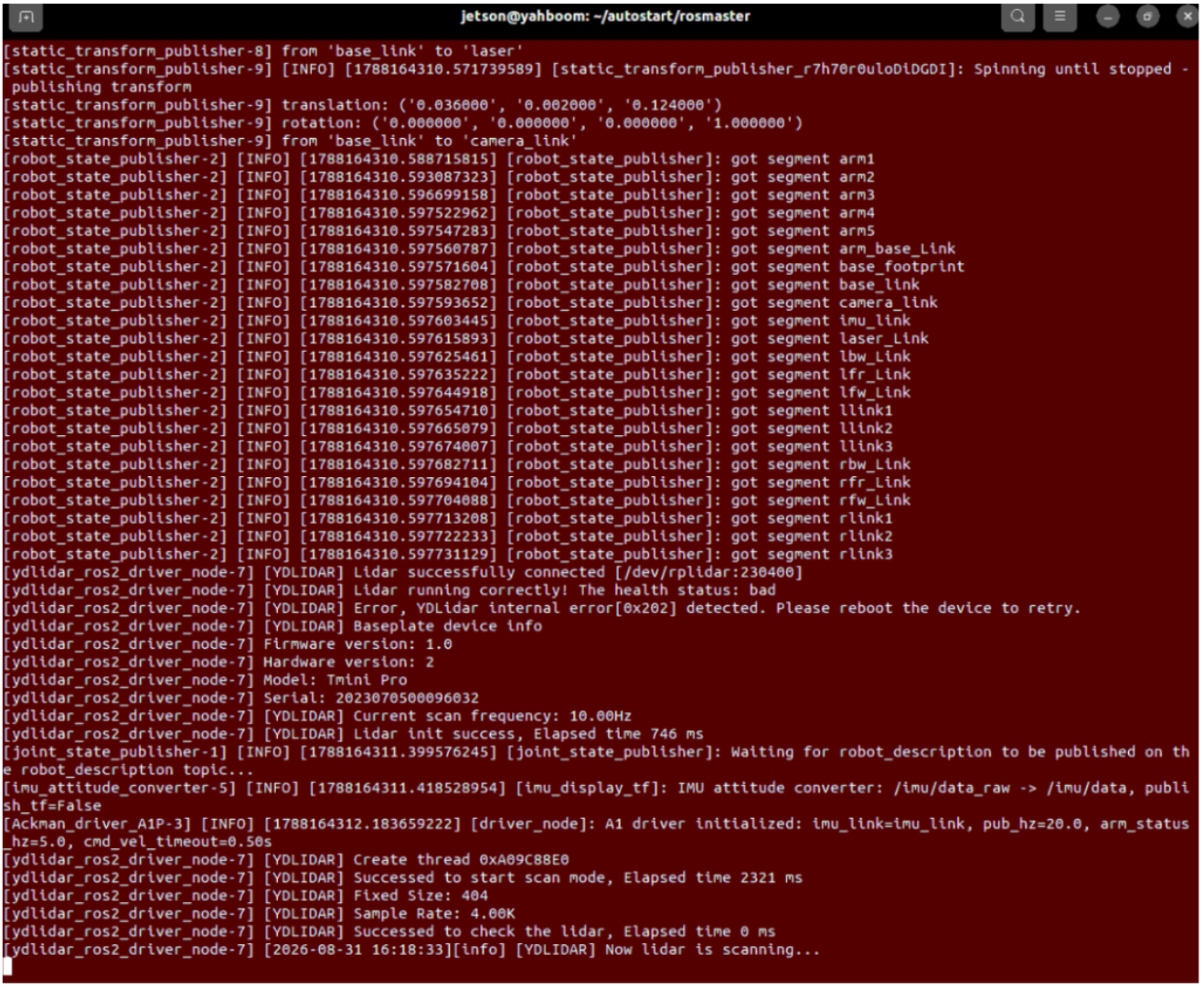
3. Preparation and Running

This example requires opening 3 terminals to start the following nodes in sequence:

3.1 Start Chassis + Lidar

Run in **Terminal 1** (one-click start of A1 Pro chassis motor driver, IMU attitude solution, and lidar):

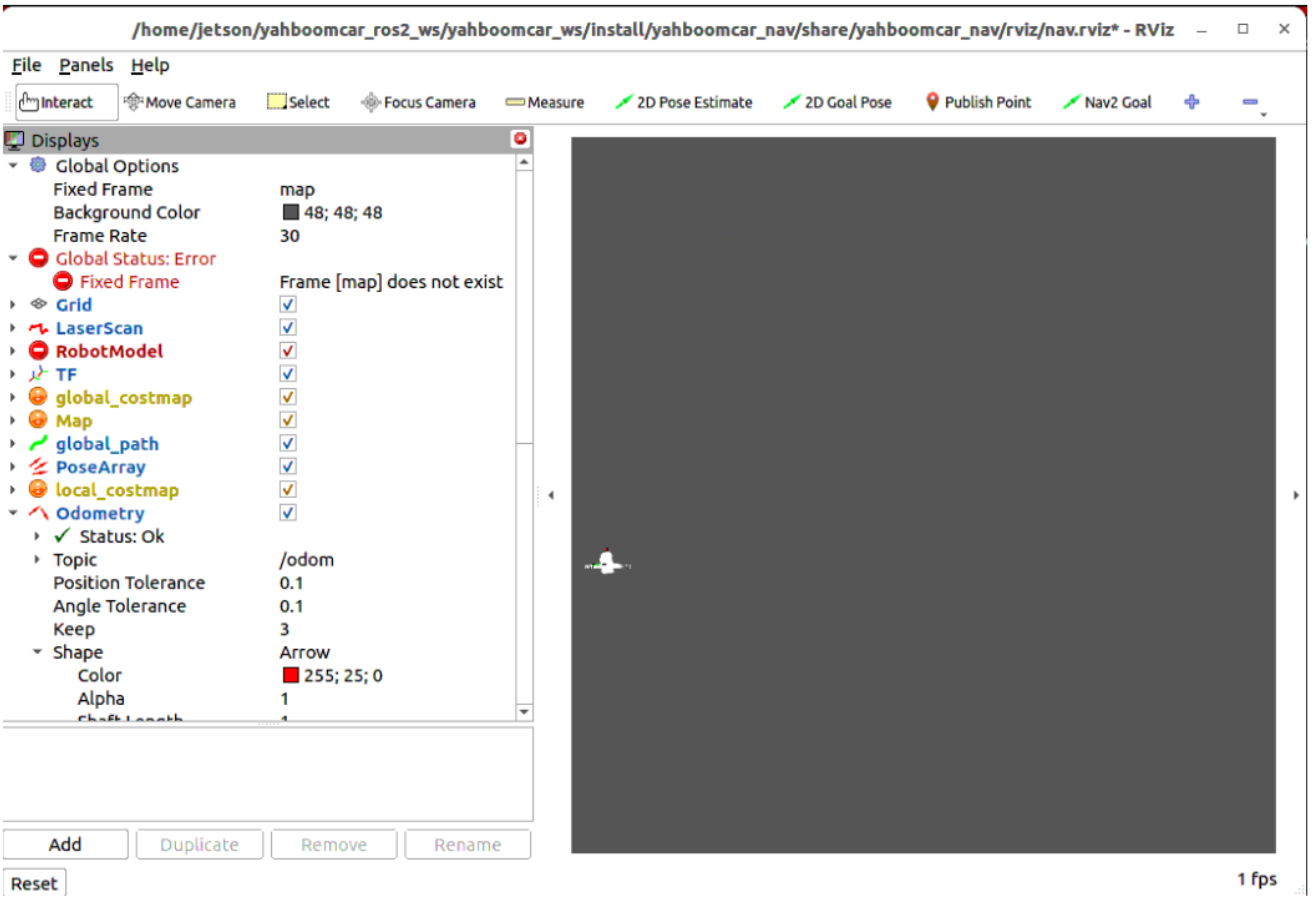
```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```



3.2 Start RViz Navigation Visualization

Run in **Terminal 2** (it is recommended to open RViz on the virtual machine if the VM and robot are on the same LAN):

```
ros2 launch yahboomcar_nav display_nav_launch.py
```



3.3 Start Nav2 TEB Navigation Core

Run in **Terminal 3** (loads default map and TEB navigation parameters):

```
ros2 launch yahboomcar_nav navigation_teb_launch.py
```

```
jetson@yahboom:~/yahboomcar_ros2_ws/yahboomcar_ws$ ros2 launch yahboomcar_nav navigation_teb_launch.py
[component_container_isolated-1] [INFO] [1788164420.877013208] [controller_server]: Controller Server has goal_checker  goal checkers available.
[component_container_isolated-1] [INFO] [1788164420.888731591] [controller_server]: Created controller : FollowPath of type teb_local_planner::TebLocalPl
annerROS
[component_container_isolated-1] [WARN] [1788164420.889626469] [rcl.logging_rosout]: Publisher already registered for provided node name. If this is due
to multiple nodes with the same name then all logs for that logger name will go out over the existing publisher. As soon as any node with that name is de
structed it will unregister the publisher, preventing any further logs for that name from being published on the rosout topic.
[component_container_isolated-1] [INFO] [1788164420.978884639] [TEBLocalPlanner]: Footprint model 'circular' (radius: 0.200000m) loaded for trajectory op
timization.
[component_container_isolated-1] [INFO] [1788164420.979029412] [controller_server]: Parallel planning in distinctive topologies enabled.
[component_container_isolated-1] [INFO] [1788164421.144136305] [controller_server]: library path : /home/jetson/yahboomcar_ros2_ws/software/library_ws/in
stall/costmap_converter/lib/libcostmap_converter.so
[component_container_isolated-1] [INFO] [1788164421.144600865] [controller_server]: Costmap conversion plugin costmap_converter::CostmapToPolygonsDBSMCCH
 loaded.
[component_container_isolated-1] [INFO] [1788164421.155189705] [controller_server]: Controller Server has FollowPath  controllers available.
[component_container_isolated-1] [INFO] [1788164421.168280134] [lifecycle_manager_navigation]: Configuring smoother_server
[component_container_isolated-1] [INFO] [1788164421.171174920] [smoother_server]: Configuring smoother server
[component_container_isolated-1] [INFO] [1788164421.181830643] [smoother_server]: Created smoother : simple_smoother of type nav2_smoother::SimpleSmoothe
r
[component_container_isolated-1] [INFO] [1788164421.192126513] [smoother_server]: Smoother Server has simple_smoother  smoothers available.
[component_container_isolated-1] [INFO] [1788164421.202792155] [lifecycle_manager_navigation]: Configuring planner_server
[component_container_isolated-1] [INFO] [1788164421.203071653] [planner_server]: Configuring
[component_container_isolated-1] [INFO] [1788164421.203172104] [global_costmap.global_costmap]: Configuring
[component_container_isolated-1] [INFO] [1788164421.211322077] [global_costmap.global_costmap]: Using plugin "static_layer"
[component_container_isolated-1] [INFO] [1788164421.213506823] [global_costmap.global_costmap]: Subscribing to the map topic (/map) with transient local
durability
[component_container_isolated-1] [INFO] [1788164421.214171838] [global_costmap.global_costmap]: Subscribing to updates
[component_container_isolated-1] [INFO] [1788164421.216802200] [global_costmap.global_costmap]: Initialized plugin "static_layer"
[component_container_isolated-1] [INFO] [1788164421.216864954] [global_costmap.global_costmap]: Using plugin "obstacle_layer"
```

> **Tip**
>
> To specify other custom maps, append the `map` parameter:

```
ros2 launch yahboomcar_nav navigation_teb_launch.py
map:=$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.yaml
```

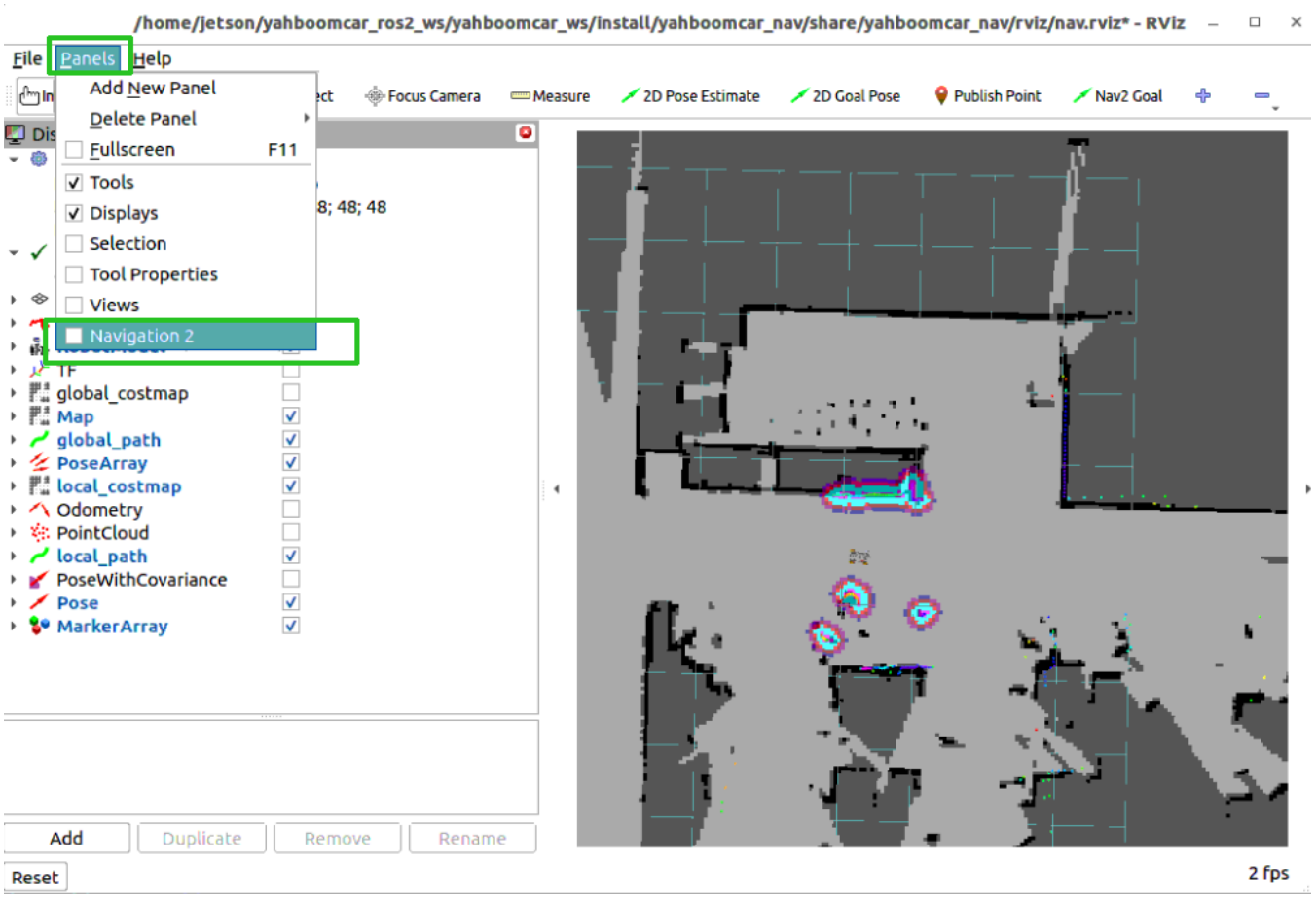
4. Multi-Point Navigation Operation Flow and Effect Description

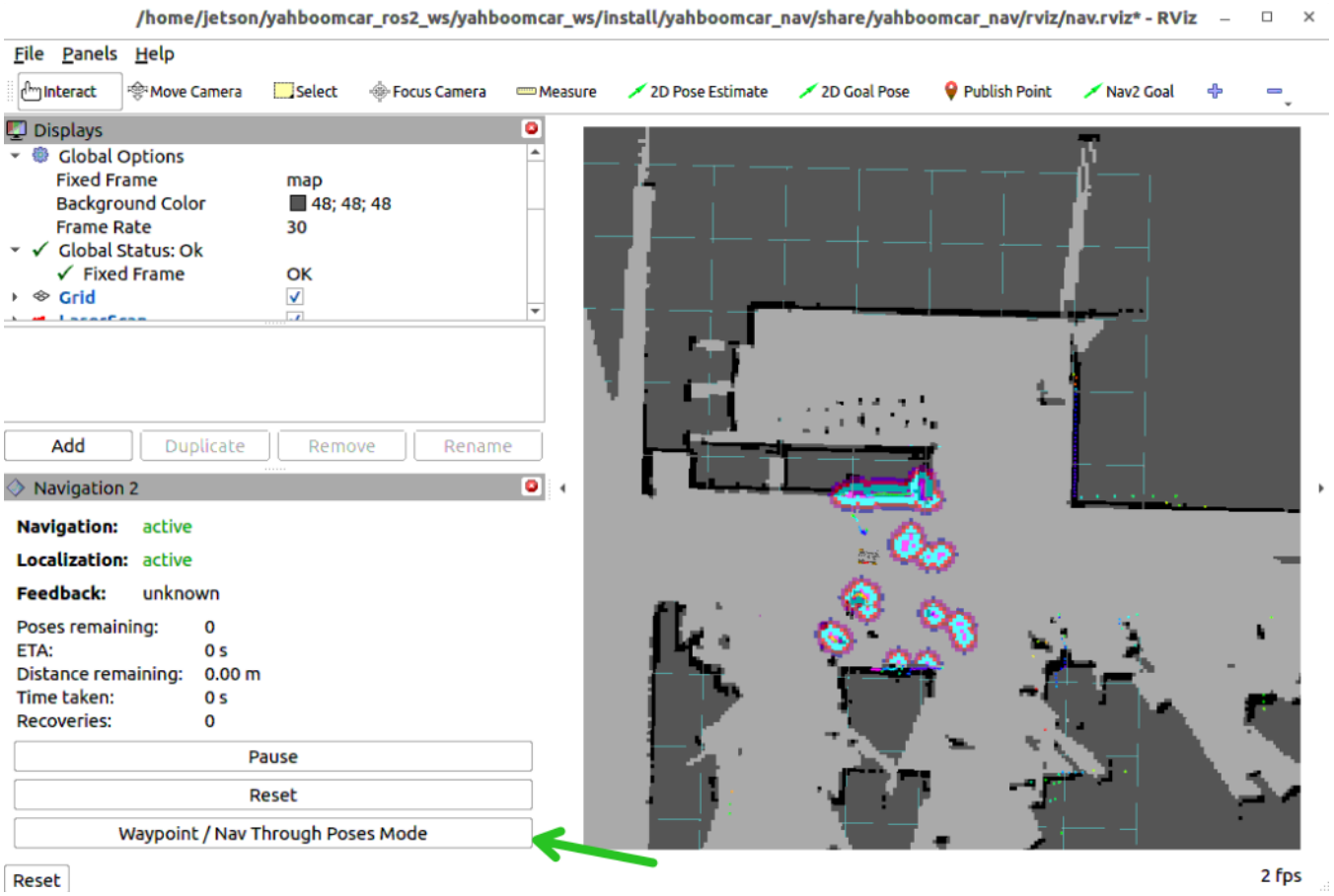
4.1 Set Initial Pose (2D Pose Estimate)

Click the `2D Pose Estimate` tool in the top toolbar of RViz2, click at the actual physical position of the robot on the map and drag out the green arrow along the chassis heading direction, aligning the laser point cloud with the black obstacle contours on the map.

4.2 Switch Waypoint Mode (Waypoint / Nav Through Poses)

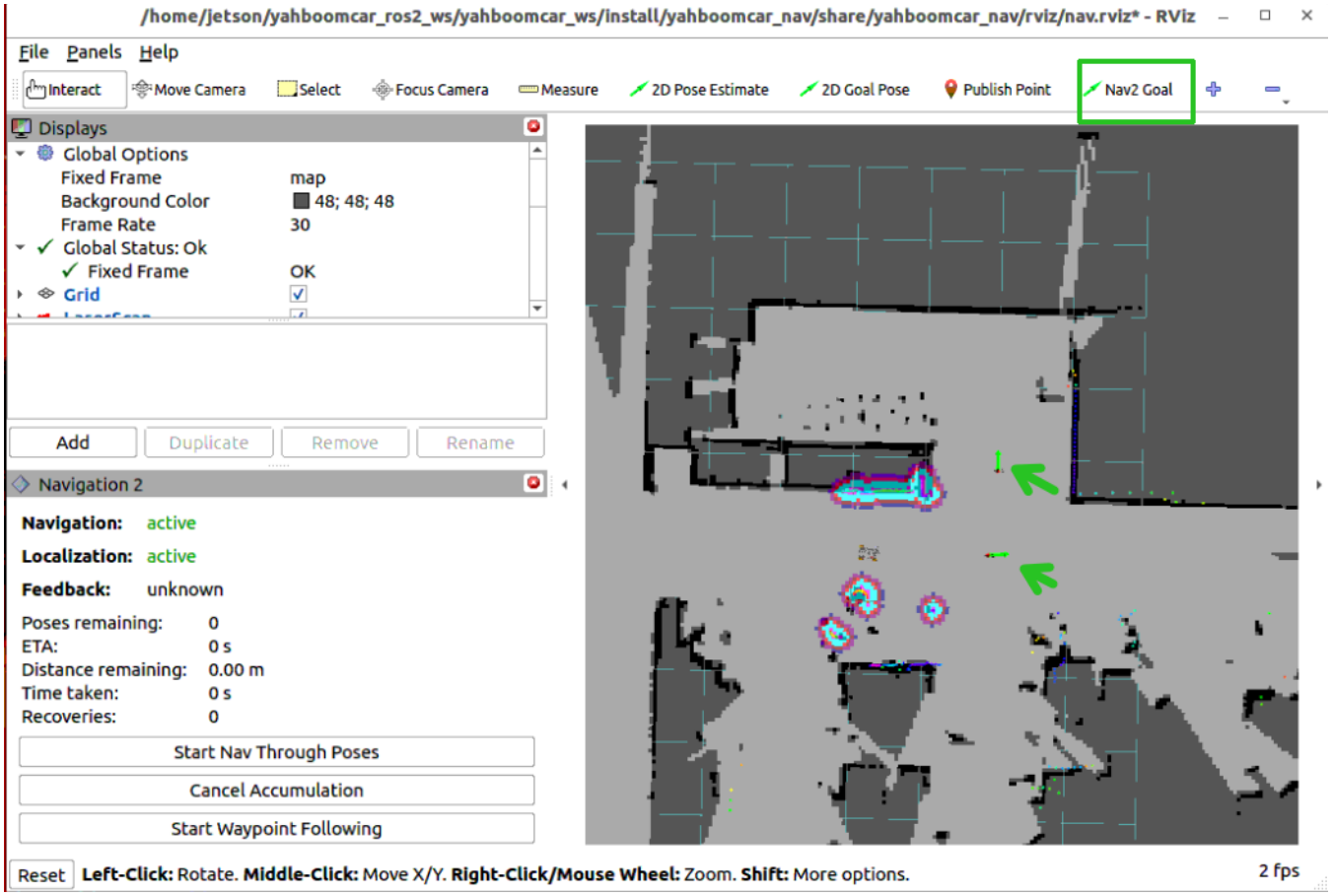
In the **Navigation2** control panel in the lower-left corner of RViz2, click the `Panels / navigation2 /Waypoint / Nav Through Poses Mode` button to switch to multi-point cruising mode.





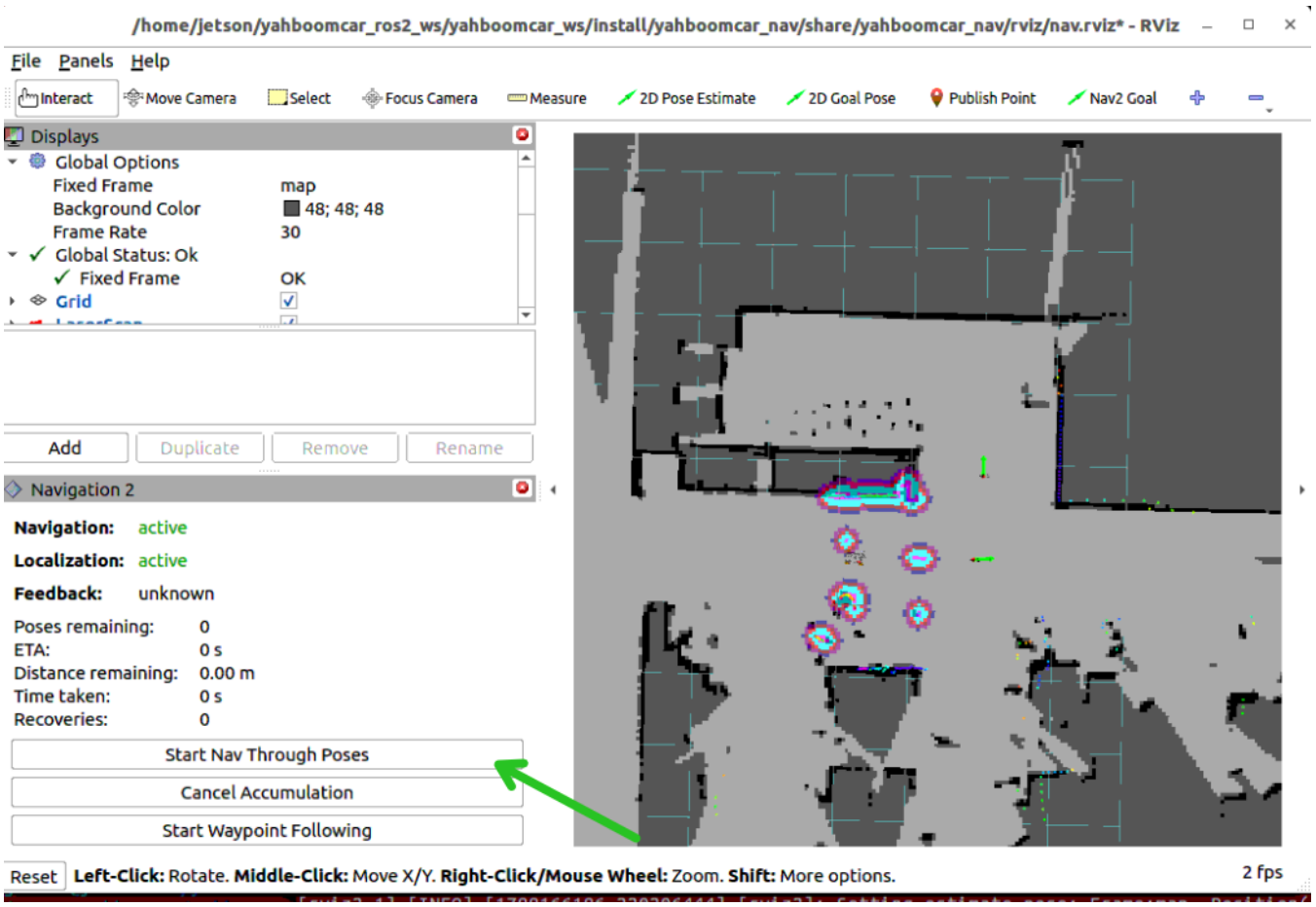
4.3 Batch Set Target Waypoints (Nav2 Goal)

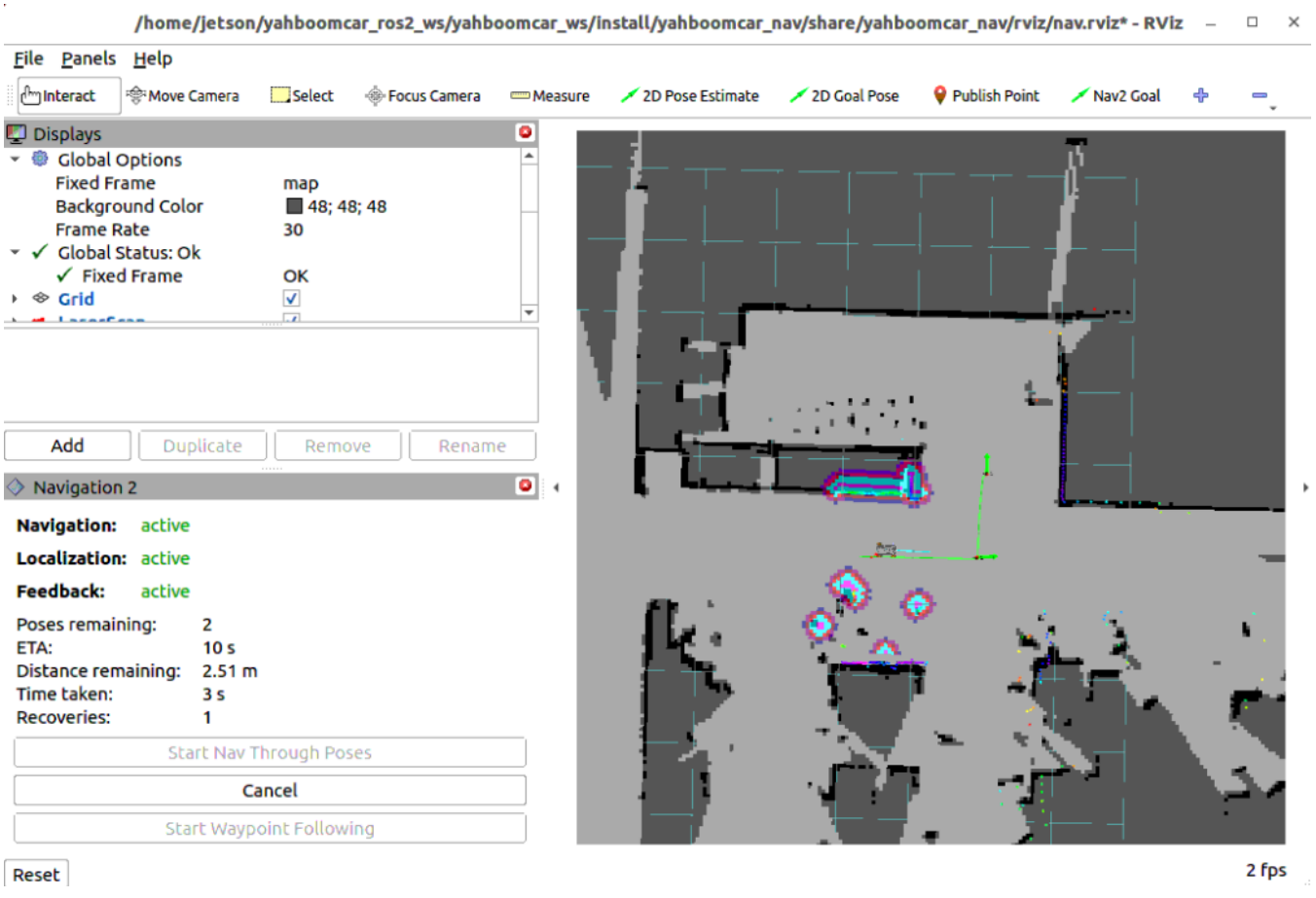
1. Click the `Nav2 Goal` (or `2D Goal Pose`) button in the top toolbar.
2. In order, click in the traversable free areas on the map and drag out heading arrows to set Point 1 (Waypoint 1), Point 2 (Waypoint 2), and Point 3 (Waypoint 3).
3. Numbered and oriented waypoint markers will be displayed on the map in sequence.



4.4 Start Multi-Point Autonomous Cruising (Start Navigation)

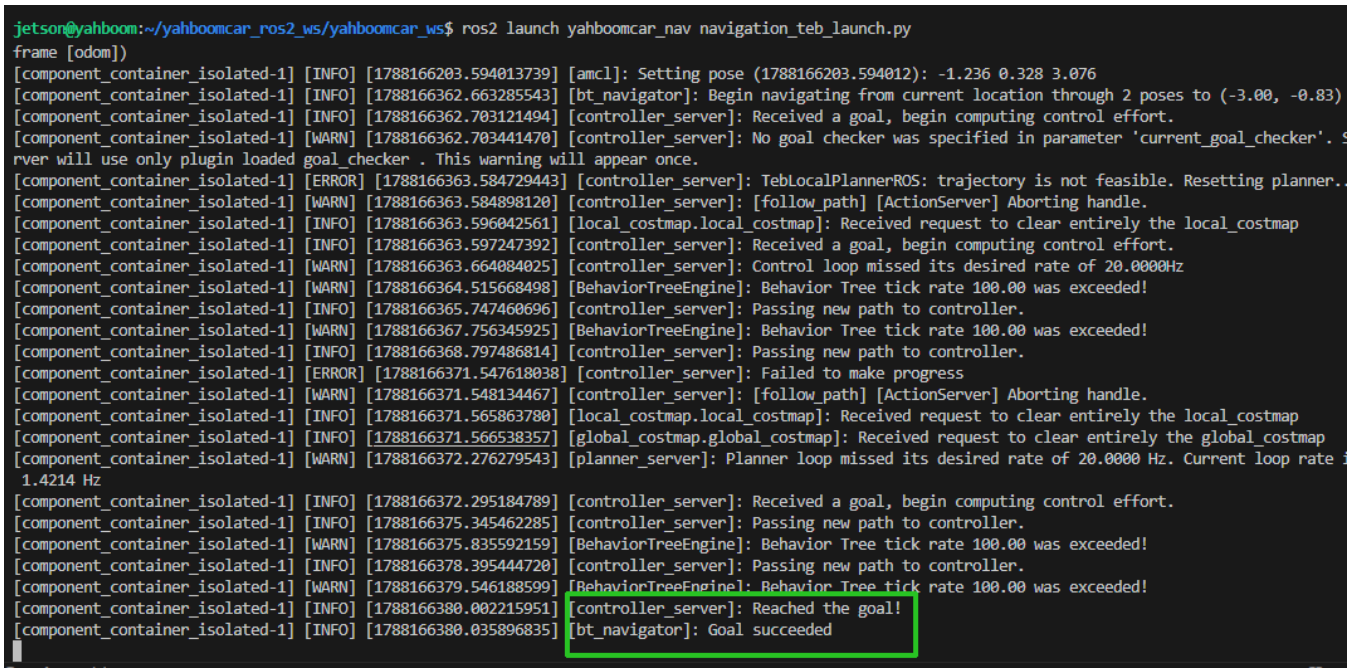
1. After waypoint calibration is complete, click `Start Waypoint Following` (or `Start Nav Through Poses`) button in the Navigation2 panel.
2. The robot immediately begins executing global path planning, automatically controlling Ackermann front-wheel steering and smooth forward movement, sequentially passing through each preset waypoint.


/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomcar_nav/share/yahboomcar_nav/rviz/nav.rviz* - RViz
File Panels Help
Interact Move Camera Select Focus Camera Measure 2D Pose Estimate 2D Goal Pose Publish Point Nav2 Goal
Displays
Global Options
Fixed Frame map
Background Color 48; 48; 48
Frame Rate 30
Global Status: Ok
Fixed Frame OK
Grid
Add Duplicate Remove Rename
Navigation 2
Navigation: active
Localization: active
Feedback: unknown
Poses remaining: 0
ETA: 0 s
Distance remaining: 0.00 m
Time taken: 0 s
Recoveries: 0
Start Nav Through Poses
Cancel Accumulation
Start Waypoint Following
Reset
Left-Click: Rotate. Middle-Click: Move X/Y. Right-Click/Mouse Wheel: Zoom. Shift: More options.
2 fps


/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomcar_nav/share/yahboomcar_nav/rviz/nav.rviz* - RViz
File Panels Help
Interact Move Camera Select Focus Camera Measure 2D Pose Estimate 2D Goal Pose Publish Point Nav2 Goal
Displays
Global Options
Fixed Frame map
Background Color 48; 48; 48
Frame Rate 30
Global Status: Ok
Fixed Frame OK
Grid
Add Duplicate Remove Rename
Navigation 2
Navigation: active
Localization: active
Feedback: active
Poses remaining: 2
ETA: 10 s
Distance remaining: 2.51 m
Time taken: 3 s
Recoveries: 1
Start Nav Through Poses
Cancel
Start Waypoint Following
Reset
2 fps

4.5 Waypoint Arrival and Terminal Feedback

When the robot sequentially arrives at all waypoints and stops steadily at the final destination, the terminal will output feedback that all goals are completed:



> **Note**
>
> To end navigation task, press `Ctrl+C` in each terminal window to stop the program.

---

5. Core Source Code and Waypoint Parameter Analysis

5.1 Multi-Point Cruising Action Communication Mechanism

Multi-point navigation is built based on ROS 2 Action mechanism:

- RViz plugin sends Action Client request containing multiple `geometry_msgs/PoseStamped` poses to Nav2 (`/navigate_through_poses` or `/follow_waypoints`).
- `bt_navigator` receives the request, calls waypoint follower (`waypoint_follower`) and planner (`planner_server`) to generate continuous trajectory, and TEB controller executes high-precision Ackermann trajectory tracking.

5.2 Waypoint Following Core Parameter Interpretation

Parameter Configuration File Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/params/teb_nav_params.yaml
```

Core Parameter Description Table

Parameter Module	Parameter Name	Current Config Value	Core Function and Physical Meaning
Waypoint Follower	`loop_rate`	`20`	Waypoint state checking and feedback update loop frequency (Hz)
Waypoint Follower	`stop_on_failure`	`false`	Whether to immediately abort entire task chain when single waypoint planning fails (`false` means skip and try next waypoint)
Waypoint Executor	`waypoint_pause_duration`	`200`	Dwell wait time at each intermediate waypoint arrival (milliseconds)
TEB Arrival Tolerance	`xy_goal_tolerance`	`0.25`	Position arrival tolerance for intermediate waypoints and final destination (meters)
TEB Arrival Tolerance	`yaw_goal_tolerance`	`0.25`	Chassis heading judgement tolerance at goal point (radians)
Speed Limit	`max_vel_x`	`0.25`	Maximum cruising linear velocity (m/s)
Steering Limit	`max_vel_theta`	`1.0`	Maximum cruising angular velocity (rad/s)

Waypoint Following Core Configuration Code Snippet

```
# Waypoint follower node configuration
waypoint_follower:
  ros__parameters:
    use_sim_time: False
    loop_rate: 20                  # State checking loop frequency (Hz)
    stop_on_failure: false         # Do not abort when a point fails, continue to next
waypoint
    waypoint_task_executor_plugin: "wait_at_waypoint"
    wait_at_waypoint:
      plugin: "nav2_waypoint_follower::WaitAtWaypoint"
      enabled: True
      waypoint_pause_duration: 200 # Waypoint dwell wait time (ms)
```

5.3 Multi-Point Calibration and Practical Operation Suggestions

1. **Avoid Calibrating Waypoints in Narrow Dead Corners**:
   Ackermann chassis requires a certain minimum turning radius; when calibrating waypoints, try to set points in the corridor center or open passages, avoiding being close to walls or inside obstacle inflation layers.

2. **Reserve Smooth Arcs for Turning Waypoints**:
   For continuous turning routes, it is recommended to calibrate 1~2 additional transition waypoints along the turning arc, helping the TEB controller generate smoother and more optimized Ackermann front-wheel steering curves.